



UNIVERSITY OF TRENTO
MASTER'S DEGREE IN DATA SCIENCE

~ · ~

2025-2026

Secure LLM orchestration for enterprise workflows using the Model Context Protocol

Supervisor
Prof. Daniele Miorandi

Graduate Student
Ettore Miglioranza
257311

FINAL EXAMINATION DATE: July 1, 2026

Dedica

Acknowledgments

Abstract

Large language models (LLMs) have demonstrated cutting-edge capabilities in understanding intent and semantic routing, but their adoption in business workflows remains limited by stringent privacy and security requirements. This thesis investigates how LLM-based orchestration can be implemented in a real-world business context, whilst providing a formal guarantee that no personally identifiable information (PII) crosses the boundaries of an external model. The work focuses on a specific workforce management domain within the Antares ERP ecosystem, where natural language requests must be reliably mapped to operational actions, such as locating nearby workers on construction sites, identifying colleagues based on proximity, and validating absence cover and substitution chains.

The central research question is: how can an external LLM be integrated as a semantic router without exposing sensitive business data, whilst ensuring deterministic and verifiable execution of business tools? To answer this question, the thesis proposes a ‘privacy-by-design’ architecture based on the Model Context Protocol (MCP). The system enforces a strict separation between language understanding and data access through a reversible anonymisation pipeline: user requests are pseudonymised before reaching the model, routed by the LLM using only anonymised text, and de-anonymised exclusively on the server side at the time of tool execution. Entity detection is performed by a multilingual NER microservice based on GLiNER, chosen over rule-based alternatives such as Microsoft Presidio precisely because of its robustness in processing business inputs in Italian. Token-to-entity mappings are stored in a short-lived Redis context, ensuring that pseudonymisation is both ephemeral and fully reversible. The MCP server exposes schema-validated, typed tools to the model and enforces strict routing constraints, whilst a .NET 10 client orchestrator manages the end-to-end request lifecycle.

The approach draws on the broader state of the art in privacy-preserving LLM infrastructure. In particular, OpenAI’s Privacy Filter (Apache 2.0, 2025), recently made open source, confirms that lightweight, on-premise NER models incorporated as pre-processing layers represent the emerging industry standard for PII containment. The key architectural contribution of this thesis relative to that reference model is *reversibility*: whilst one-way encoding destroys the mapping between placeholders and entities, the token-based scheme proposed here preserves full operational correctness whilst allowing controlled de-anonymisation at the protocol boundary, a distinction relevant under Article 4.5 of the GDPR, which classifies such processing as pseudonymisation rather than anonymisation.

The thesis makes three concrete contributions: (1) a multi-service architecture that structurally isolates sensitive data from the LLM context, (2) a deterministic and reversible privacy layer based on token mapping that preserves the correctness of business logic throughout the entire request lifecycle, and (3) a validated set of realistic workforce management use cases demonstrating the effectiveness of the approach in a representative production environment. The validation confirms that the accuracy of semantic routing and NER performance on Italian-language inputs are sufficient for enterprise deployment and that no raw PII is observable within the boundaries of the tested LLM context.

This work positions MCP as a practical and applicable interface standard for enterprise AI orchestration and highlights the key engineering trade-off inherent in privacy by design: increased architectural complexity is the necessary cost for demonstrable privacy assurance, but it unlocks reliable and verifiable LLM-driven workflows without violating data protection constraints.

Keywords: LLM orchestration, Model Context Protocol (MCP), privacy-by-design, PII

pseudonymization, semantic routing, named entity recognition, GLiNER, enterprise AI security, GDPR compliance

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	The Problem of Data Privacy in LLM Integration	1
1.3	Research Objectives and Questions	1
1.4	Contributions	1
1.5	Structure of the Thesis	1
2	Background and Related Work	3
2.1	Large Language Models in Enterprise Workflows	3
2.1.1	Semantic Routing and Intent Classification	4
2.1.2	Tool Calling and AI Agents	4
2.2	The Model Context Protocol (MCP)	5
2.2.1	Standardizing LLM Interfaces	5
2.2.2	Security Boundaries in MCP	5
2.3	Privacy-Preserving Natural Language Processing	6
2.3.1	Personally Identifiable Information (PII) Definitions	6
2.3.2	Named Entity Recognition (NER) for Data Masking	6
2.3.3	Rule-based vs. Deep Learning Approaches (Presidio vs. GLiNER)	7
2.4	State of the Art in Secure LLM Orchestration	7
2.4.1	OpenAI Privacy Filter: A Reference Open-Weight Model for PII Redaction	8
3	System Architecture and Design	11
3.1	Architectural Principles: Privacy-by-Design	11
3.2	The Antares AI Ecosystem	12
3.3	High-Level System Architecture	12
3.4	Component Design	14
3.4.1	The MCP Client Orchestrator	14
3.4.2	The Anonymization Service API	14
3.4.3	The GLiNER-based NER Microservice	15
3.4.4	The Antares MCP Server	15
3.5	Data Flow and Lifecycle of a Request	16
3.5.1	Tokenisation and Ephemeral Context Storage	16
3.5.2	Isolated Semantic Routing	16
3.5.3	Server-Side Deanonymisation and Tool Execution	16

4	Implementation Details	19
4.1	Technology Stack and Development Environment	19
4.2	The Reversible Anonymisation Pipeline	20
4.2.1	Early Approach: Microsoft Presidio and its Limitations	20
4.2.2	The Pivot to GLiNER for Multilingual Italian NER	20
4.2.3	Deterministic Token Mapping and Ephemeral Context Storage	21
4.3	Exposing Business Operations via MCP	22
4.3.1	C# Attributes and Reflection for Tool Schema Generation	22
4.3.2	The <code>ask_antares</code> Routing Tool	23
4.3.3	Strict Prompting for Privacy Enforcement	24
4.4	Use Case Orchestration in <code>JobTools</code>	24
4.4.1	Use Case A: Nearest Workers to a Cantiere	24
4.4.2	Use Case B: Colleagues of a Person	25
4.4.3	Use Case C: Holiday Request Analysis	25
4.5	Backend Integration and Data Abstraction	26
4.5.1	The <code>IMiorelliService</code> Interface	26
4.5.2	The Stub/Mock Pattern	27
4.5.3	NSwag Client Generation and the Migration Path	28
5	Use Cases and Validation	29
5.1	Workforce Management Domain Overview	29
5.2	Use Case A: Spatial Querying for Construction Sites	29
5.3	Use Case B: Colleague Identification and Proximity Ranking	29
5.4	Use Case C: Holiday Request and Absence Analysis	29
6	Discussion and Future Work	31
7	Conclusions	33
7.1	Summary of the Work	33
7.2	Final Remarks on Enterprise AI Adoption	33
	Glossary	33
	Nomenclature list	33
	Bibliography	36
	List of Figures	37
	List of Tables	39

Chapter 1

Introduction

1.1 Context and Motivation

1.2 The Problem of Data Privacy in LLM Integration

1.3 Research Objectives and Questions

1.4 Contributions

1.5 Structure of the Thesis

Chapter 2

Background and Related Work

This chapter reviews the three bodies of literature that inform the design of the system presented in subsequent chapters: the integration of large language models into enterprise software, the Model Context Protocol as a standardisation layer for LLM tool interfaces, and privacy-preserving natural language processing with a focus on named entity recognition and PII protection. Section 2.1 examines how large language models change semantic routing and tool invocation in enterprise systems, and the engineering constraints that shift introduces. Section 2.2 turns to the Model Context Protocol and the security boundaries it defines between client, server, and model. Section 2.3 addresses the regulatory and technical grounding of PII protection, from the GDPR’s definition of personal data to detection approaches ranging from rule-based systems to zero-shot transformers. Section 2.4 surveys the current state of the art in secure LLM orchestration and identifies the gap that motivates this work.

2.1 Large Language Models in Enterprise Workflows

The modern generation of large language models traces its architectural lineage to the transformer, introduced by Vaswani et al. [18] in 2017. The attention mechanism, which allows each token in a sequence to attend to every other token, proved to be the basis for a qualitative leap in language understanding. Scaling transformer-based models, exemplified by GPT-3 [2] with its 175 billion parameters, showed that sufficiently large models exhibit capabilities not present in smaller counterparts: multi-step reasoning, few-shot generalisation from in-context examples, and reliable instruction following across diverse domains [19].

For enterprise software, the practical implication is a new paradigm for human-system interaction. Until recently, bridging the gap between a user’s natural language expression and a system’s formal data operations required rule engines, supervised intent classifiers, or custom NLP pipelines. A general-purpose language model can close that gap from the prompt alone, interpreting natural language requests and mapping them to well-typed business operations without labelled training data or domain-specific engineering.

The shift creates real engineering problems. Enterprise workflows demand deterministic, auditable outputs. Sensitive business data must not be transmitted to third-party model providers. System behaviour must remain predictable under adversarial or malformed inputs. These constraints sit in direct tension with the generative, probabilistic nature of LLMs, and reconciling them is an active area of research and engineering.

2.1.1 Semantic Routing and Intent Classification

Semantic routing refers to interpreting a natural language utterance and directing it to the appropriate downstream component based on its inferred intent. Classical approaches used supervised intent classifiers trained on labelled utterance datasets [9]. These models required curated training data per domain and were brittle under paraphrasing, multilingual input, or out-of-distribution requests. In practice, that limitation has a direct cost: adding a new business action, or supporting a new language, means collecting and labelling a fresh batch of example utterances before the classifier can be retrained and redeployed, a cost that scales with every intent and every language a deployment needs to cover.

LLMs remove that cost. Because they acquire broad linguistic knowledge during pre-training, they can classify intent zero-shot given only a natural language description of the available actions, with no labelled dataset and no retraining step. Routing logic becomes a prompt rather than a trained artefact, and adding a new action or language is a matter of editing text, not collecting data. A recent example, though outside classic intent classification, is a conversational, RAG-based assistant for AI-supported document creation workflows [5], which replaces a rule-based document generation flow with natural-language interaction end-to-end. The saving is not free, however. The quality of zero-shot routing depends heavily on the specificity and clarity of tool descriptions, and vague or overlapping descriptions cause the model to route incorrectly regardless of the underlying model's capability, making prompt engineering a first-class concern in deployed systems.

2.1.2 Tool Calling and AI Agents

The capacity for LLMs to invoke external functions (also called function calling) was formalised as a first-class API feature by major model providers in 2023. In this paradigm, the developer supplies the model with a schema describing callable functions, their parameters and return types. The model produces a structured call rather than free text; the host application executes it and feeds the result back for the model to incorporate into its response.

The ReAct framework [20] formalised the interleaving of reasoning traces and tool invocations, showing that a model that explicitly reasons about which tool to call outperforms models that invoke tools without intermediate reasoning, especially in multi-step tasks. This is the AI agent paradigm: a language model acting as an orchestrator that selects and sequences tool calls to reach a higher-level goal.

The Toolformer work [17] demonstrated that models can learn to use external APIs from self-supervised examples, acquiring tool use directly from patterns in text rather than from an explicit interface definition. Nothing in that training signal constrains the resulting call to a fixed, machine-checkable format: a malformed or incomplete invocation is not distinguishable, from the model's own output, from a well-formed one. That gap is not cosmetic in an enterprise setting. A routing decision that cannot be validated against a schema cannot be logged in a structured, replayable form, and there is no reliable way to confirm after the fact which tool was called with which parameters. For this reason, enterprise deployments generally prefer explicit schema-driven invocation over learned tool use. Schema-driven invocation provides stronger output format guarantees and makes the routing decision auditable.

Two architectural properties of tool-calling systems have direct implications for enterprise deployments. First, the model's agency can be bounded by restricting which tools it may call: exposing only business-level operations and keeping data access inside deterministic code limits the model to intent classification and parameter extraction. Second, the tool

result feedback loop, in which tool outputs are returned to the model for further reasoning, is the primary vector for indirect prompt injection attacks [8], making its design a security-relevant decision.

2.2 The Model Context Protocol (MCP)

The Model Context Protocol [1], published by Anthropic in November 2024, standardises the communication interface between LLM client applications and external tool servers. Before MCP, each LLM-tool integration required bespoke glue code, custom serialisation formats, and proprietary conventions. MCP defines a vendor-neutral interface so that any compliant client can connect to any compliant server without integration-specific adaptation.

The design is modelled after the Language Server Protocol (LSP) [3], which eliminated the $N \times M$ explosion of editor-language integrations by decoupling language-analysis servers from editor clients. MCP applies the same logic: any client that speaks MCP can talk to any server that speaks MCP, reducing the integration surface from $N \times M$ to $N + M$.

2.2.1 Standardizing LLM Interfaces

MCP defines three categories of server-exposed primitives. *Tools* are callable functions with typed input schemas and structured return values, analogous to OpenAPI operations. *Resources* are read-only data sources the model can pull into its context: file contents, database records, configuration objects. *Prompts* are reusable, parameterised message templates.

Communication runs over two transports: standard input/output for locally spawned server processes, and HTTP with Server-Sent Events (SSE) for network-accessible servers. The underlying message format is JSON-RPC 2.0. Tool schemas are JSON Schema objects describing parameter types, constraints, and documentation. This standardised schema representation allows tooling to auto-generate type-safe client code from server definitions, reducing the risk of schema drift between client expectations and server implementation.

2.2.2 Security Boundaries in MCP

MCP’s security model separates three roles. The *MCP host* manages client connections and controls which servers the model may reach. The *MCP server* is a trusted executor with access to real data and infrastructure. The *model* is a semantic reasoner: it produces structured tool calls but accesses no data source directly.

This three-layer separation creates a natural enforcement point for privacy. The server is the only component that handles raw sensitive data. A pre-processing step applied to user requests before they reach the model can structurally prevent PII from entering the LLM context, because the server’s trust boundary sits downstream of the model. The model operates only on what the client chooses to send.

Prompt injection [8] is the primary known attack on this architecture. Malicious content embedded in tool results or externally retrieved resources can attempt to hijack the model’s behaviour by simulating system-level instructions. Mitigations include restricting the model’s tool set, limiting the scope of requests the model may issue, and routing business logic through deterministic code that does not re-enter the model after execution. These mitigations reduce the attack surface but do not eliminate it, and the design of the tool result feedback loop remains an open security concern in MCP deployments.

2.3 Privacy-Preserving Natural Language Processing

User requests directed at business systems routinely contain personally identifiable information: employee names, site addresses, phone numbers, absence dates. Under data protection regulations, transmitting this information to external model providers requires a valid legal basis and appropriate safeguards. This section reviews the regulatory context and the technical approaches developed to address PII exposure in NLP pipelines.

2.3.1 Personally Identifiable Information (PII) Definitions

The General Data Protection Regulation [7], Article 4(1), defines personal data as “any information relating to an identified or identifiable natural person.” Identifiability is broad: a person is identifiable if they can be singled out by name, identification number, location data, or factors specific to their physical, professional, or social identity. Combinations of individually innocuous data points can identify an individual in aggregate.

Article 4(5) introduces *pseudonymisation*: processing personal data so that it can no longer be attributed to a specific person without additional information, provided that additional information is kept separately with appropriate technical safeguards. Pseudonymisation is distinct from anonymisation. Pseudonymised data remains personal data under the GDPR, because the mapping between pseudonyms and identities is preserved, even if stored separately. Truly anonymous data cannot be re-linked by any reasonably likely means and falls outside the regulation entirely.

This distinction has direct engineering implications for privacy-preserving LLM pipelines. A system that replaces PII with reversible tokens before sending data to an external model provider is performing pseudonymisation, not anonymisation. The privacy guarantee it provides is a limitation of PII exposure to the model, not its elimination. The token-to-entity mapping must itself be protected and handled as personal data under the GDPR.

2.3.2 Named Entity Recognition (NER) for Data Masking

Named entity recognition is the NLP task of identifying and classifying contiguous spans of text that refer to named entities: persons, organisations, locations, dates, phone numbers. For PII protection, NER is the detection layer of a two-stage pipeline: entities are identified, then replaced with neutral placeholders that preserve syntactic structure without revealing identifying content.

Classical NER systems based on Conditional Random Fields or bidirectional LSTMs required annotated corpora per language and domain. Pre-trained transformer encoders, BERT [6] and its multilingual variants, reduced data requirements substantially by enabling fine-tuning on small labelled datasets. Domain and language specificity remained a persistent challenge. A model trained on English newswire won’t reliably handle Italian business communications.

The error asymmetry in masking pipelines is architecturally significant. A false negative (a missed PII span) directly violates the privacy guarantee: the unmasked entity crosses the model boundary in plaintext. A false positive (a masked non-PII span) degrades the utility of the downstream task but creates no privacy breach. This asymmetry implies that detection components should be optimised for recall, with post-processing to suppress common false positives.

The design of replacement tokens also affects downstream task quality. Replacing every entity with a single generic token (e.g., [REDACTED]) destroys structural information the

model may need: if a request mentions two different people and both become the same token, the model cannot distinguish them when making a routing decision. Label-typed, numbered tokens such as <PERSON_0001> and <PERSON_0002> preserve entity type and cardinality while removing identifying content, enabling correct multi-entity routing even when all PII has been replaced.

2.3.3 Rule-based vs. Deep Learning Approaches (Presidio vs. GLiNER)

Two broad approaches are available for PII detection in NLP pipelines: rule-based systems that combine regular expressions, gazetteers, and lightweight statistical models; and transformer-based systems that apply encoder models to span classification over arbitrary text.

Microsoft Presidio [12] is the leading open-source rule-based framework. It provides recognisers for names, phone numbers, email addresses, credit card numbers, and national identifiers, built on regular expressions and context-sensitive matching, optionally extended with spaCy NER models. Its strengths are predictability, low computational cost, and an extensible plugin architecture. Its weaknesses are language dependence (most recognisers target English) and rigidity: entity types that don't match predefined patterns are silently missed, with no indication of failure. Extending coverage to a new language or entity type requires writing explicit rules, which is labour-intensive and inherently incomplete.

GLiNER [21] (Generalist Model for Named Entity Recognition) is a zero-shot NER model built on a bidirectional transformer encoder. Instead of training a fixed classification head per entity type, GLiNER conditions detection on natural language label descriptions given at inference time. The model computes a similarity score between the label description and each candidate span, enabling detection of arbitrary entity types without fine-tuning. The multilingual variant `urchade/gliner_multi-v2.1`, trained on data spanning multiple European languages, provides coverage of morphologically rich languages such as Italian, including proper name patterns and domain-specific vocabulary that rule-based approaches miss.

Compared to Presidio, GLiNER offers three concrete advantages for multilingual and domain-diverse deployments. The detection label set is configurable at runtime without model retraining. Italian-language support is native, not an adaptation. Each prediction carries a confidence score, enabling a configurable threshold to tune the precision-recall trade-off. A post-processing pipeline over raw GLiNER predictions (span normalisation, stopword filtering, overlap deduplication) can further improve precision without sacrificing recall on genuine PII.

2.4 State of the Art in Secure LLM Orchestration

The dominant pattern in privacy-sensitive LLM deployments is a *privacy-preserving proxy*: a pre-processing layer that sanitises user input before it reaches the model. Early implementations used regular expression substitution, adequate for structured PII (phone numbers, email addresses) but inadequate for unstructured natural language. More recent work applies transformer-based NER, trading computational cost for substantially better recall on diverse inputs.

Three systems anchor the current state of the art for PII detection in LLM pipelines: Microsoft Presidio, the rule-based baseline discussed in Section 2.3.3; GLiNER, the zero-shot transformer detector adopted in the architecture presented in this thesis; and OpenAI's Privacy Filter, reviewed in detail below as the most complete open-weight alternative. A related

concern shows up in adjacent academic work: Mansillo [10] anonymises synthetic training data inside a multimodal retrieval-augmented pipeline, which confirms that anonymisation-aware LLM systems are an active area of interest beyond this thesis, though that work does not address reversibility or tool-execution correctness either. The three systems diverge along the dimensions that matter for an enterprise LLM orchestration system, and no single one of them covers all of them.

All three run entirely on local infrastructure and ship under a permissive licence, MIT for Presidio and Apache 2.0 for the other two, so neither dimension separates them.

Multilingual coverage is uneven, however. Presidio’s recognisers are English-centric and need new rules for each additional language, GLiNER’s multilingual variant handles Italian natively, and Privacy Filter’s non-English performance is undocumented and, per its own model card, may degrade on morphologically rich languages. Reversibility separates them further. Presidio can be reversible in principle through a dedicated encrypt/decrypt operator pair, but that ties reversibility to key management external to the detection pipeline. GLiNER is a detector with no built-in notion of a token-to-entity mapping; reversibility, where it exists, is a property of the system built around it, not of GLiNER itself. Privacy Filter performs one-way redaction by design and discards the mapping entirely. On detection accuracy, Privacy Filter is the only one of the three to publish a comparable accuracy figure, discussed in detail in Section 2.4.1. Presidio and GLiNER have no comparably reported number. Presidio is tuned for precision over broad recall by construction, and GLiNER is tuned for recall across arbitrary label sets, so neither figure would be comparable to Privacy Filter’s self-reported one even if it existed.

None of the three systems combines reversibility with a validated benchmark for operational correctness. The one system with a quantitative accuracy figure, Privacy Filter, is also the one that discards the token-to-entity mapping outright. The systems that could in principle preserve or approximate that mapping, Presidio and GLiNER, report no comparable benchmark of their own.

The literature addresses detection and masking largely in isolation. End-to-end systems that detect PII, mask it before the model boundary, execute business operations using real values server-side, and return a safe response to the client remain uncommon in published work. Most published systems stop at the masking step and do not address the operational requirement that tool execution needs the original entity values, not their placeholders.

2.4.1 OpenAI Privacy Filter: A Reference Open-Weight Model for PII Redaction

The most significant open-weight contribution to this space is OpenAI’s Privacy Filter [14], released in April 2026 under the Apache 2.0 licence. It is the first publicly available, production-quality PII detection model explicitly designed as a pre-processing layer for LLM pipelines.

Architecture. Privacy Filter is a bidirectional token-classification model derived from the GPT OSS family. The autoregressive generation head is replaced by a classification head over a BIOES span-labelling scheme with 33 output classes. A constrained Viterbi decoder enforces coherent span boundaries across multi-token entities. The Sparse Mixture-of-Experts design gives 1.5 billion total parameters but only approximately 50 million active per token, enabling high-throughput inference at low cost.

Capabilities. The model detects eight PII categories: `private_person`, `private_address`, `private_email`, `private_phone`, `private_url`, `private_date`, `account_number`, and `secret`. Its context window is 128,000 tokens. On the PII-Masking-300k benchmark it achieves 96%

F1 (97.43% on the corrected version). Fine-tuning on small in-domain datasets can push domain-specific performance from 54% to 96% F1 quickly, suggesting a strong general-purpose foundation.

Local execution. The model is designed to run on-premise, within the organisation’s infrastructure, removing the need for a Data Processing Agreement with a cloud vendor and addressing GDPR data residency requirements directly.

One-way redaction. Privacy Filter performs one-way redaction: the placeholder tokens it produces carry no mapping back to the original entities. This is appropriate for use cases where the model’s output does not need to reference original PII values, such as summarisation, classification, and general question-answering. For enterprise workflows in which a routing decision must trigger operations on real, identified data, one-way redaction is insufficient. A system that routes a request mentioning `<LOCATION_0001>` must know what `<LOCATION_0001>` is at the point of tool execution.

OpenAI’s model card acknowledges that Privacy Filter performs pseudonymisation under GDPR Article 4(5), not anonymisation. The placeholder tokens combined with the original document could in principle be reversed by a party holding both. This is the correct framing of what such systems achieve: they limit PII exposure to the model provider, but the mapping between tokens and entities still exists and must be managed.

Multilingual limitations. Privacy Filter’s performance on non-English languages is not officially documented and may degrade significantly on morphologically rich languages such as Italian.

The body of work reviewed in this chapter establishes that privacy-preserving pre-processing is the right pattern for LLM orchestration in sensitive enterprise domains. What the existing systems do not address is the combination of reversibility with operational correctness: enabling a routing system to work on pseudonymised inputs while guaranteeing that tool execution uses the original entity values, without exposing those values to the model provider. This gap is the starting point for the architecture described in Chapter 3.

Chapter 3

System Architecture and Design

This chapter presents the architecture of Antares AI, a privacy-preserving LLM orchestration system for enterprise workforce management. The design is governed by a single non-negotiable invariant: *no personally identifiable information crosses the LLM context boundary in cleartext*. Every architectural decision described here is a consequence of enforcing that invariant.

3.1 Architectural Principles: Privacy-by-Design

Privacy by design, formalised by Cavoukian [4] and later codified as a legal obligation in GDPR Article 25 [7], is the principle that privacy protection must be built into systems from the outset. Cavoukian’s framework identifies seven foundational principles: proactive prevention of privacy harms, privacy as the default operational setting, privacy functionality embedded into the architecture, full functionality without zero-sum trade-offs, end-to-end security across the data lifecycle, visibility and transparency in system operation, and user-centric design. The Antares AI architecture applies these principles as structural design constraints, not as runtime policy.

Three of Cavoukian’s principles shape the system most directly.

Privacy as the default. Anonymisation of user requests before LLM transmission is the only supported operational path. A client that skips the anonymisation step sends an unprocessed prompt, but the server system prompt still instructs the model to treat input as pseudonymised tokens; bypassing anonymisation is detectable and requires an explicit decision. There is no configuration flag to disable privacy enforcement.

Privacy embedded into design. PII isolation is not achieved through audit logging, access controls, or post-hoc output redaction. It is structural: by placing the anonymisation service at the architectural boundary, PII is physically absent from the data that crosses it. The model cannot leak what it never received.

End-to-end security. The token-to-entity mapping is stored in an ephemeral, session-scoped Redis context and consumed exactly once during server-side deanonymisation. No component persists PII beyond the minimum lifetime required for a single business operation. Operational reports written to disk by the `JobTools` layer stay on server-local storage and are never sent to the client.

Together these constraints define the *architectural invariant*: PII may exist in the system, but only within the server’s trusted execution boundary, and only for the duration of one business operation.

3.2 The Antares AI Ecosystem

Antares AI was developed within the Antares/Miorelli ecosystem, an ERP platform for workforce management used in Italian construction and facility management. The platform manages personnel assignments to job sites (*cantieri*), tracks absence and leave requests, and maintains the hierarchical relationships between workers and managers. Users currently interact via structured web interfaces; Antares AI enables the same operations through natural language, routing each request to the correct business tool automatically.

The domain is demanding on privacy and correctness for concrete reasons. The data is personnel-sensitive by nature: names, phone numbers, residential coordinates, absence records, and employment relationships are all PII under the GDPR. The user base is multilingual (requests arrive in Italian, regional dialects, or a mix), which ruled out recogniser-based NER tools designed for English. Most critically, the operational context requires accurate results: a holiday request analysis that produces an incomplete result is operationally harmful, not just inconvenient, because it may leave a construction site under-staffed. The system must preserve full business logic accuracy even when anonymising inputs.

Workforce operations are structured around three core entity types: *persone* (workers), *cantieri* (construction sites), and *ticket* (absence and leave records). Relations are typed: `PULITO_DA` links a worker to their assigned sites, `RESPONSABILE_DI_RIFERIMENTO` identifies a worker’s manager, and `AUTORIZZATORE` identifies the person authorised to approve leave. These entity types and relation names appear in natural language requests and define the PII surface the anonymisation pipeline must cover.

3.3 High-Level System Architecture

Antares AI comprises four cooperating runtime services, each with a distinct responsibility and a well-defined interface. Figure 3.1 illustrates the runtime topology and data flows.

The four services and their roles are summarised in Table 3.1.

Table 3.1: Antares AI runtime services

Service	Stack	Port	Role
MCP Client	C#/.NET 10	—	Request orchestration and test harness
Anonymization API	C#/.NET 10	5292	PII detection, tokenisation, context storage
GLiNER Service	Python/FastAPI	5100	Named entity recognition
MCP Server	C#/.NET 10	5001	LLM routing and business tool execution

The communication topology mirrors the system’s trust model. The MCP client is an untrusted endpoint: it never receives real personnel data, and the only thing it sees is a hardcoded Italian confirmation string. The anonymisation API sits at the client-side boundary, processing the raw prompt and issuing a session token (`contextId`) that indexes the stored mapping. The GLiNER service is a stateless NER oracle; it knows nothing about tokens, Redis, or business context. The MCP server is the trusted execution boundary: it receives only anonymised prompts, routes them via Claude, restores PII server-side, and executes business tools against real data.

Redis [16] runs at port 55000 as the ephemeral context store. It is not reachable from the client and is accessible only to the anonymisation API (writing) and the MCP server (reading and consuming).

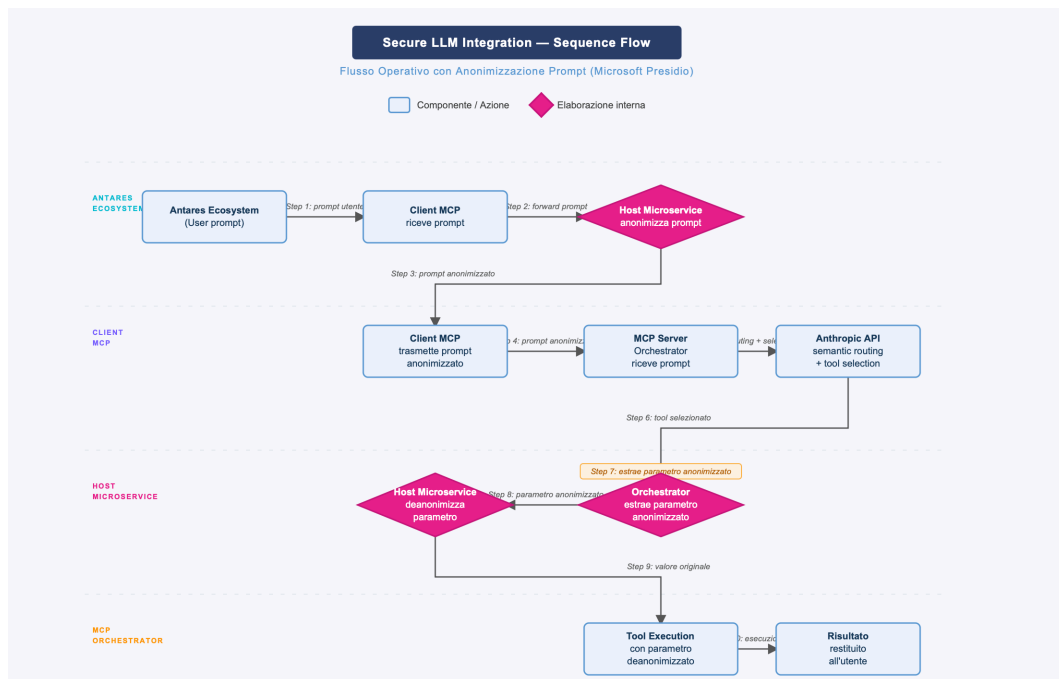


Figure 3.1: Runtime architecture of Antares AI. The client anonymises the user prompt before crossing the MCP boundary. The MCP server routes the anonymised request via Claude and executes the selected business tool server-side, where PII is restored from the Redis context. The client receives only a hardcoded confirmation string.

The architectural invariant (no PII crosses the MCP boundary) is enforced at the interface between client and server. The `ask_antares` tool accepts an anonymised prompt and a `contextId`; it never receives raw PII. The client calls the anonymisation API first, gets an anonymised prompt and a `contextId`, and only then constructs the tool call. PII restoration from the `contextId` happens inside the server’s trusted boundary, after the routing decision has been made.

3.4 Component Design

3.4.1 The MCP Client Orchestrator

The MCP client (`Antares_AI_McpClient`) is a .NET 10 console application that orchestrates the end-to-end request lifecycle. In the current system it serves as both the reference client implementation and the integration test harness for the three use case scenarios.

Its responsibilities are deliberately minimal. It constructs a natural language prompt, submits it to the Anonymization API, and then calls `ask_antares` on the MCP server with the anonymised prompt and `contextId`. The client performs no business logic and holds no sensitive data; it can’t observe PII because it never calls a deanonymisation endpoint or touches the Redis context.

Communication with the MCP server uses the HTTP/SSE transport via `ModelContextProtocol.Core`. Communication with the Anonymization API uses a strongly typed HTTP client generated by NSwag [13] from the API’s OpenAPI specification, keeping the client’s view of the API interface in sync with the server implementation automatically.

A production client would accept dynamic user input, manage multi-turn context across `contextId` sessions, and surface token placeholders in the UI as opaque references. The current implementation uses hardcoded prompts to enable deterministic integration testing.

3.4.2 The Anonymization Service API

The Anonymization Service (`AnonymizationServiceAPI`) is an ASP.NET Core Minimal API application that acts as the PII gateway between client and server. It exposes three REST endpoints: `GET /anonymize`, `POST /anonymize-into-context`, and `POST /deanonymize-known`.

The primary client-facing operation is `/anonymize`. It receives a raw prompt, delegates entity detection to the GLiNER service, replaces detected entities with stable typed tokens, generates a `contextId`, stores the token-to-entity mapping in Redis, and returns the anonymised text, the `contextId`, and a detection count. The token format is `<LABEL_NNNN>`, where `LABEL` is the uppercase entity type and `NNNN` is a zero-padded counter per label per context. This preserves entity type and cardinality while stripping identifying content.

The `/anonymize-into-context` endpoint extends an existing session by merging new token mappings into an already-issued `contextId`. This supports multi-turn interactions where a follow-up request introduces a new entity that must be pseudonymised within the same operational context.

The `/deanonymize-known` endpoint is the server-facing operation. It receives an anonymised string and a `contextId`, retrieves the mapping from Redis via a `Pop` operation (atomic read-and-delete), and restores original values through string substitution. The one-time-use design limits the exposure window for the mapping to the minimum needed for tool execution; a repeated call with the same `contextId` will find no context and fail cleanly.

Detection is fully delegated to the GLiNER service. The separation between detection

(GLiNER) and masking plus context management (Anonymization API) preserves substitutability: the NER backend can be replaced without touching the tokenisation logic.

3.4.3 The GLiNER-based NER Microservice

The GLiNER service (`GLiNERService`) is a Python FastAPI [15] application that exposes a single stateless endpoint, `POST /analyze`, and a health check at `GET /health`. Its job is to apply the GLiNER model to an input text and return a list of detected entity spans with labels, character offsets, and confidence scores.

The multilingual model `urchade/gliner_multi-v2.1` [21] is loaded once at startup and held in memory for the process lifetime. Model weights download from HuggingFace on first run and are cached locally; subsequent starts use the cached copy.

The service is a dumb NER oracle. It receives text and labels, runs the model, returns spans. All policy decisions (which labels to request, what confidence threshold to apply, how to handle overlapping spans) are made by the Anonymization API, which configures the `/analyze` call accordingly. This keeps the GLiNER service simple and reusable.

Three post-processing stages run within the Anonymization API before token substitution. Raw predictions are normalised: invalid offsets, missing labels, and out-of-bounds spans are discarded. Low-quality entities are filtered out: spans below the 0.45 confidence threshold, spans shorter than three characters, and spans matching an Italian stop-word blocklist are removed. Overlapping predictions are deduplicated, keeping the higher-confidence, longer span. The result is a clean, non-overlapping list of entity spans ready for replacement.

3.4.4 The Antares MCP Server

The MCP server (`Antares_AI_McpServer`) is the trusted execution boundary. It exposes the `ask_antares` routing tool, calls the Claude API for semantic routing, and executes the selected business tool against the Miorelli data layer.

Tools are registered via `ModelContextProtocol.AspNetCore`, which generates MCP-compliant schemas from C# class and method annotations. Two types are registered: `AntaresTools` (exposing `ask_antares`) and `JobTools` (exposing the three business operations). The `JobTools` methods are not called directly by the MCP client; Claude invokes them through function-calling within an `ask_antares` execution.

The server calls Claude Sonnet 4.6 via the `Anthropic.SDK` client. The system prompt defines Claude’s role as a semantic router, lists the available sub-tools with descriptions, and enforces privacy constraints: the model must not infer or reveal PII from tokens, must forward the `contextId` to the selected sub-tool, and must include the sub-tool’s result string verbatim in its response. A retry loop with exponential backoff handles transient `overloaded_error` responses, retrying up to four times before returning an error message.

The data access layer is abstracted behind `IMiorelliService`. The current implementation registers `MiorelliServiceStub`, which throws `MiorelliClientNotAvailableException` on every call; each business tool catches this and falls back to static mock data from `MiorelliMockDataProvider`. Switching to the real Miorelli tenant requires changing one line in `program.cs`: registering `AntaresMiorelliClient` instead of the stub. No tool code changes. This design is discussed in Chapter 4.

3.5 Data Flow and Lifecycle of a Request

This section traces a user request from natural language prompt to confirmation response. Use Case A (finding the nearest available workers to a named construction site) serves as the concrete example.

3.5.1 Tokenisation and Ephemeral Context Storage

The lifecycle begins when the MCP client constructs a natural language prompt. For Use Case A, a typical prompt is: *“Trova i lavoratori più vicini al cantiere Bergamo Via della Fonda 5 Giulia S.r.l. entro 15 km. Risolvi tutti i GUID internamente.”* This contains two PII spans: a location name (*Bergamo Via della Fonda 5*) and a company name (*Giulia S.r.l.*).

The client calls `GET /anonymize`. The API calls the GLiNER service, receives entity predictions, runs the three-stage filtering pipeline, and sorts surviving entities by character offset in reverse order. Replacements are applied right-to-left to avoid invalidating offsets of earlier spans. The location becomes `<LOCATION_0001>` and the company becomes `<ORGANIZATION_0001>`.

The mapping `{"<LOCATION_0001>": "Bergamo Via della Fonda 5", "<ORGANIZATION_0001>": "Giulia S.r.l."}` is stored in Redis under `anon:context:{contextId}`, where `contextId` is a freshly generated UUID. The API returns the anonymised prompt and the `contextId` to the client. No PII is in the response; the client holds only an opaque session identifier.

Redis is the right tool here for practical reasons: its in-memory architecture gives sub-millisecond read and write latency, which matters in a synchronous request pipeline. Its native hash type matches the token-map structure directly, and its in-memory-with-optional-persistence model suits ephemeral session data that does not need to survive a service restart.

3.5.2 Isolated Semantic Routing

The client constructs the `ask_antares` tool call with the anonymised prompt and `contextId`, dispatching it over HTTP/SSE. At this point the MCP boundary has been crossed and no PII is in transit.

The `ask_antares` implementation builds a Claude API call. The system prompt defines Claude’s role as an Italian-language semantic router, provides the JSON schemas of the available sub-tools, and specifies the expected response format. The anonymised user prompt becomes the user message.

Claude’s task is intent classification (which sub-tool?) and parameter extraction (what values, in anonymised form?). For Use Case A it classifies the intent as `get_nearest_workers_to_cantiere` and extracts `cantiereName = "<LOCATION_0001>"` and `context_id = {contextId}`. It doesn’t try to resolve the token; it works entirely with the pseudonymised representation.

The Claude API returns a structured tool call rather than free text. The `Microsoft.Extensions.AI` abstraction intercepts it, locates the registered `GetNearestWorkersToCantiere` method, and invokes it with the extracted parameters. The Redis context is not touched during this phase.

3.5.3 Server-Side Deanonymisation and Tool Execution

The `GetNearestWorkersToCantiere` method is the first point where PII re-enters the system. Its first action is to call `DeanonymizeAsync`, which posts to `/deanonymize-known` with the token `"<LOCATION_0001>"` and the `contextId`. The API atomically reads and deletes

the context from Redis and returns the original value: "Bergamo Via della Fonda 5". The Redis context no longer exists after this call.

The deanonymised value drives the business logic entirely within the server's trusted boundary. The method resolves the caller's authorised cantieri via the Miorelli service layer, locates the matching site, retrieves assigned workers via the PULITO_DA relation, computes Haversine distances from each worker's registered coordinates to the site's GPS coordinates, and filters and ranks results by distance within the specified radius.

The full report, containing real names, phone numbers, distances, and status information, is written to a timestamped file in the server's local `logs/` directory. It is never transmitted to the client. The method returns a hardcoded Italian confirmation string: *"Operazione completata con successo."* Claude includes this verbatim in its final response.

The client receives a short text block with no PII. The invariant holds across the entire lifecycle: no personally identifiable information crossed the MCP boundary, and a complete, accurate operational result was produced server-side.

Chapter 4

Implementation Details

This chapter describes the concrete implementation of the Antares AI system, progressing from technology stack selection through the development of each major subsystem. Where Chapter 3 described the system’s structure and the invariants it enforces, this chapter describes how those invariants are realised in code: what language constructs, libraries, and design patterns are used to achieve the architectural goals under the practical constraints of the development context.

4.1 Technology Stack and Development Environment

The system is implemented across two programming ecosystems: C#/.NET 10 [11] for the three services that form the trusted execution boundary (MCP client, Anonymization API, MCP server), and Python with FastAPI [15] for the GLiNER NER microservice. The choice of two ecosystems reflects two distinct requirements that could not be simultaneously satisfied by a single language.

The C# ecosystem was chosen for the core services because the Antares/Miorelli business platform is built on a .NET stack, and the internship mandate included contributing to an existing codebase. .NET 10, the current long-term support release at the time of development, provides mature support for ASP.NET Core Minimal APIs, dependency injection, typed HTTP clients, and the NSwag OpenAPI toolchain. The `ModelContextProtocol.AspNetCore` package (the reference .NET implementation of the MCP server transport) is also a .NET library, making C# the natural fit for the MCP server.

Python was chosen for the GLiNER service because the GLiNER model is distributed as a Python package and the HuggingFace ecosystem, which provides the underlying transformer infrastructure, is primarily Python-native. FastAPI was selected as the HTTP framework for its automatic OpenAPI schema generation, Pydantic-based request validation, and async-first design, which is well-suited to a single-endpoint microservice with potentially variable inference latency.

Redis [16] serves as the sole infrastructure dependency beyond the four application services. Running on port 55000 to avoid conflicts with the default Redis port, it provides the ephemeral key-value store for token-to-entity context maps. Redis’s in-memory data model provides the sub-millisecond read/write latency required for the anonymisation step to be non-blocking within the synchronous request path.

Dependency management follows the conventions of each ecosystem. .NET packages are declared in `.csproj` files and resolved automatically by `dotnet restore` at build time; no

manual installation step is required. Python dependencies are declared in `pyproject.toml` and managed by `uv`, a modern Python package manager that combines dependency resolution and virtual environment management. API client code is generated from OpenAPI specifications using NSwag [13]: the Anonymization API exports a YAML spec that NSwag processes to generate strongly typed C# client classes in both the MCP server and the MCP client projects, eliminating manual HTTP client maintenance.

4.2 The Reversible Anonymisation Pipeline

The anonymisation pipeline is the technical core of the system’s privacy guarantee. Its implementation went through two distinct phases: an initial approach based on Microsoft Presidio that proved insufficient for the target domain, and a redesigned pipeline built around GLiNER that satisfied the recall requirements of the operational context.

4.2.1 Early Approach: Microsoft Presidio and its Limitations

The initial implementation used Microsoft Presidio [12] as the PII detection engine. Presidio’s appeal lay in its maturity: it provides a well-documented Python SDK with built-in recognisers for common PII categories, a clear separation between detection (`AnalyzerEngine`) and replacement (`AnonymizerEngine`), and an extensible architecture that supports custom recognisers via the `EntityRecognizer` base class.

In practice, Presidio’s performance on the actual Italian-language business inputs received from the company proved inadequate. The built-in NER models underlying Presidio’s `SpacyRecognizer` are trained primarily on English corpora; Italian named entity recognition, particularly for person names with regionally specific morphological patterns and construction site addresses, exhibited recall rates well below the threshold required for a reliable privacy guarantee. Attempts to compensate with custom pattern-based recognisers improved precision on predictable formats (email addresses, Italian fiscal codes) but had no effect on free-text person names embedded in natural language sentences. The fundamental limitation is structural: rule-based systems require explicit coverage of every entity pattern, and the diversity of Italian-language business vocabulary defies comprehensive enumeration.

The decision to replace Presidio was taken after the company provided the concrete use case inputs that the system would be evaluated against. With real prompts in hand, the gap between Presidio’s output and the required entity coverage was unambiguous, and a redesign of the detection layer was the only viable path to meeting the privacy invariant.

4.2.2 The Pivot to GLiNER for Multilingual Italian NER

The replacement of Presidio with GLiNER [21] required a redesign of the detection microservice from the ground up. The GLiNER service is implemented as a standalone FastAPI application exposing a single `POST /analyze` endpoint that accepts a text string, a list of entity labels, a confidence threshold, and a language hint, and returns a list of detected entity spans with their labels, character offsets, and confidence scores.

Model selection followed from the multilingual requirement. The `urchade/gliner_multi-v2.1` checkpoint, a multilingual fine-tune of the GLiNER architecture trained on data spanning multiple European languages, was selected for its Italian-language coverage and its ability to handle the domain-specific vocabulary of Italian construction and workforce management. The model is loaded once at application startup and held in a module-level global, avoiding

per-request loading overhead and ensuring that the first request does not incur an anomalous latency spike.

The entity labels submitted to the model at inference time are configured in the Anonymization API's `appsettings.json` under the GLiNER section and passed through to the GLiNER service on each call:

```

1 "GLiNER": {
2   "BaseUrl": "http://localhost:5100",
3   "TimeoutSeconds": 30,
4   "MinScore": 0.45,
5   "Labels": [
6     "person", "organization", "location", "address",
7     "email", "phone number", "date", "time"
8   ]
9 }
```

Listing 4.1: GLiNER entity label configuration in `appsettings.json`

This configuration externalises the detection policy from the model itself. Adjusting which entity types to detect, or changing the confidence threshold, requires only a configuration file update with no code change or model retraining.

The raw GLiNER predictions pass through a three-stage filtering pipeline in the Anonymization API before token substitution runs. The first stage is normalisation: spans with invalid character offsets, missing labels, or lengths below three characters are dropped. The second is low-quality filtering: spans below the 0.45 confidence threshold and spans whose normalised text matches an Italian stopword blocklist are removed. The blocklist targets words GLiNER occasionally misclassifies as named entities: common prepositions, articles, and domain-specific imperative verbs like *ritorna* or *risolvi*. The third stage deduplicates overlapping predictions: candidates are sorted by confidence then by span length, and a greedy pass discards any prediction whose character range overlaps an already-selected one. The output is a clean, non-overlapping, position-sorted list ready for substitution.

4.2.3 Deterministic Token Mapping and Ephemeral Context Storage

Token generation follows a deterministic scheme: each detected entity is assigned a token of the form `<LABEL_NNNN>`, where `LABEL` is the uppercase entity type (`PERSON`, `LOCATION`, `ORGANIZATION`, etc.) and `NNNN` is a zero-padded four-digit counter, incremented per label per context and starting from 0001. A prompt containing two distinct person names therefore produces `<PERSON_0001>` and `<PERSON_0002>`, which the LLM can distinguish as separate entities even though neither token reveals the underlying name.

Token substitution is applied right-to-left over the sorted list of entity spans. This ordering is critical for correctness: replacing a span near the end of the string does not shift the character positions of earlier spans, so each offset from the GLiNER output remains valid for the duration of the substitution loop. A left-to-right approach would require offset adjustment after each replacement, introducing complexity and potential for off-by-one errors.

The resulting token-to-entity map is stored in Redis under the key `anon:context:{contextId}`, where `contextId` is a freshly generated UUID issued for the current request. The storage interface is defined by the `IAnonymizationContextStore` contract, implemented by

RedisAnonymizationContextStore:

```

1 public interface IAnonymizationContextStore
2 {
3     Task Store(string contextId,
4               Dictionary<string, string> tokenMap);
5     Task<Dictionary<string, string>> Get(string contextId);
6     Task Append(string contextId,
7               Dictionary<string, string> newTokens);
8     Task<Dictionary<string, string>> Pop(string contextId);
9 }

```

Listing 4.2: IAnonymizationContextStore contract

The `Pop` method is the critical operation for privacy enforcement: it reads the token map from Redis and *atomically deletes* the context entry in a single operation. This one-time-use semantic ensures that the PII mapping is consumed and destroyed at the first deanonymisation call, which occurs inside `JobTools` immediately before business logic execution. Any subsequent call to `Pop` with the same `contextId` returns a not-found response, preventing replay attacks or residual data accumulation in the context store. No time-to-live (TTL) is currently set on context keys; they persist until consumed by `Pop` or until a Redis restart. In a production deployment, a conservative TTL (e.g., five minutes) should be applied to bound the exposure window in the event of a failed or abandoned request.

The `Append` method supports multi-turn interactions: when a follow-up user message introduces a new named entity, the client calls `POST /anonymize-into-context` with the existing `contextId`, and the new token mappings are merged into the existing context without overwriting prior entries. This allows a single `contextId` to accumulate the full PII surface of a multi-turn conversation.

4.3 Exposing Business Operations via MCP

4.3.1 C# Attributes and Reflection for Tool Schema Generation

The MCP specification requires each registered tool to be accompanied by a JSON Schema describing its input parameters, including names, types, descriptions, and optionality. Writing and maintaining these schemas manually is error-prone and creates a risk of drift between the schema the model sees and the actual method signature. The `ModelContextProtocol.AspNetCore` library solves this by generating schemas automatically from C# class and method annotations using runtime reflection.

Tool classes are decorated with `[McpServerToolType]` at the class level and `[McpServerTool(Name = "...")]` at the method level. The library discovers all annotated methods at application startup, infers the parameter schema from the method's C# parameter types and XML documentation comments, and registers the resulting tool descriptors with the MCP transport layer. The following excerpt illustrates the annotation pattern for the `ask_antares` entry point:

```

1 [McpServerToolType]
2 public class AntaresTools
3 {
4     [McpServerTool(Name = "ask_antares",
5                   Description = "Semantic router for Antares AI. " +

```



```

6         "Pass the anonymised user request and the contextId.")]]
7     public async Task<string> AskAntares(
8         string user_input,
9         string? context_id = null)
10    { ... }
11 }

```

Listing 4.3: MCP tool annotation on AntaresTools

The `Description` property on `[McpServerTool]` is surfaced to the LLM as the tool’s documentation in the function invocation schema. Clear, unambiguous descriptions are therefore a direct input to the routing quality: if the description of a sub-tool does not accurately convey its purpose, the model may route requests incorrectly. The descriptions used in the production configuration are written in English (matching the MCP schema convention) and use precise operational language: each sub-tool description specifies the input it expects, the entity type it resolves, and the relation type it traverses.

Tool classes are registered with the ASP.NET Core dependency injection container as scoped services and declared to the MCP framework via `.WithTools<T>()`, which causes the framework to invoke reflection at startup and register the discovered schemas. All internal tool classes (`JobTools`, `DepartmentsTools`, `PersonTools`, `AssetTools`, `AbsenceTools`, `LookupTools`) are also registered as scoped services, allowing the MCP-exposed tool classes to receive them by constructor injection.

4.3.2 The `ask_antares` Routing Tool

The `ask_antares` method is the single entry point through which all client requests enter the trusted execution boundary. Its implementation proceeds in three phases: system prompt construction, Claude API invocation with function invocation enabled, and response extraction.

The system prompt is constructed as a structured Italian-language message that establishes Claude’s role and operational constraints. It identifies the model as the semantic routing layer of the Antares workforce management system, instructs it to select exactly one sub-tool from the enumerated list based on the user’s intent, requires it to forward the `context_id` parameter to the selected sub-tool unchanged, and prohibits it from inferring, revealing, or reconstructing any PII from the anonymised tokens in the user’s prompt. The privacy constraint is stated explicitly: the model is instructed to treat tokens such as `<PERSON_0001>` as opaque identifiers and to never attempt to deduce the identity they represent.

The Claude API is invoked via the `Microsoft.Extensions.AI` abstraction layer using `IChatClient.GetResponseAsync()` with the `ChatOptions.UseFunctionInvocation = true` flag. This flag enables the function invocation loop: the client sends the system prompt, user message, and the tool schemas to the model; the model emits a structured tool call; the abstraction layer executes the corresponding C# method; and the method’s return value is fed back to the model for final response synthesis. The client then extracts `response.Text` as the tool’s return value, which includes the hardcoded confirmation string produced by the invoked sub-tool.

A retry mechanism handles the `overloaded_error` response that the Claude API may return under high load. The implementation retries up to four times with exponential backoff starting at two seconds, doubling on each attempt (2s, 4s, 8s, 16s). After all retries are exhausted, the method returns an Italian-language error message to the client indicating

that the service is temporarily unavailable. The retry sequence is implemented inline rather than using a general-purpose resilience library to keep the dependency surface minimal.

4.3.3 Strict Prompting for Privacy Enforcement

The system prompt is the primary mechanism for enforcing the privacy invariant at the LLM layer. Three constraints are embedded alongside the operational instructions.

The model is prohibited from including real names, phone numbers, addresses, or any other identifying information in its response. The only content it may return is the confirmation string emitted by the selected sub-tool. Even if the model could infer entity identities from parametric knowledge, the prompt explicitly forbids their appearance in the output.

Anonymised tokens must be preserved verbatim in any parameter the model extracts and forwards. If the user prompt contains `<PERSON_0001>`, the model passes the literal string `<PERSON_0001>` to the sub-tool, not a guessed or hallucinated name. This ensures the token reaches `JobTools` intact for deanonymisation.

The prompt also explicitly frames the operational context as an enterprise business system with data protection obligations. Instruction-following models comply more reliably with privacy constraints when the prompt explains why they exist, even if that understanding is entirely prompt-derived.

4.4 Use Case Orchestration in JobTools

The `JobTools` class implements the high-level orchestration logic for the three business use cases. Each method follows the same five-step pattern: deanonymise the relevant parameter, resolve the caller’s identity and required GUID lookups, execute the business logic via the Miorelli service layer, build and persist a full real-data report to a server-local log file, and return a hardcoded Italian confirmation string. The deanonymisation step is always the first action, ensuring that PII re-enters the system only at the start of trusted server-side execution.

4.4.1 Use Case A: Nearest Workers to a Cantiere

Use Case A (`get_nearest_workers_to_cantiere`) finds the workers assigned to a named construction site and ranks them by proximity to the site’s GPS coordinates within a configurable radius.

The deanonymised cantiere name is used to resolve the target site from the caller’s list of authorised cantieri. Resolution requires three prior lookups: the caller’s identity (to establish the perimeter of authorised sites), the GUID of the `PULITO_DA` relation type (to identify workers assigned to the site), and the GUID of the `SedePrincipale` item category (to filter cantieri by type). These GUIDs are tenant-specific constants in the Miorelli platform and are resolved at runtime via `LookupTools.GetRelationTypes` and `GetItemCategories`.

Distance computation uses the Haversine formula, which calculates great-circle distance between two points on a sphere given their latitude and longitude in decimal degrees:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos \phi_1 \cos \phi_2 \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

where ϕ denotes latitude, λ denotes longitude, $r = 6,371$ km is the mean Earth radius, and d is the great-circle distance between the two points. The formula is applied

in `PersonTools.CalculateDistanceAndRank`, a pure computation method with no service calls, which ranks a list of `MiorelliPerson` records by their distance from a reference point and optionally returns only the top- N closest. Workers without registered coordinates are sorted to the end of the list rather than discarded, preserving the completeness of the result. A subsequent `FilterByRadiusKm` call removes workers whose distance exceeds the requested radius.

The ranked list is partitioned into *attivi* (active workers) and *non attivi* (inactive workers, identified by the `IsActive` flag on `MiorelliPerson`) and written to a timestamped log file in the server's `logs/` directory. The log includes full names, phone numbers, distances in kilometres, and activity status. The confirmation string “*Operazione completata con successo.*” is returned regardless of the number of workers found; the operational report is the authoritative output, not the LLM response.

4.4.2 Use Case B: Colleagues of a Person

Use Case B (`get_colleagues_of_person`) identifies the direct colleagues of a named worker (those sharing at least one cantiere via the `PULITO_DA` relation) and supplements this list with the five nearest non-colleagues, ranked by proximity to the target person's residential coordinates.

The deanonymised person name is resolved to a `MiorelliPerson` record via `PersonTools.SearchPersonByName`, which performs a case-insensitive substring match across the full name field of the mock dataset. The direct colleague set is computed by `PersonTools.GetColleaguesOfPerson`: for each cantiere to which the target person is assigned, all `PULITO_DA` relations are retrieved, and the union of the resulting person sets (minus the target person) constitutes the colleague set. Each `MiorelliColleague` record includes the shared cantiere, enabling the report to state not only who the colleagues are but which sites they share.

The non-colleague pool is constructed by collecting all workers from all cantieri accessible to the caller, removing the target person and all identified colleagues. Haversine distances are computed from the target person's residential coordinates to each candidate, and the five closest are selected. Ranking by residential proximity is a deliberate design choice for the workforce management domain: in the context of substitute identification, proximity to the absent worker's habitual work area is a meaningful operational signal.

4.4.3 Use Case C: Holiday Request Analysis

Use Case C (`report_holiday_request`) is the most complex orchestration, combining an authorisation check with a per-cantiere absence analysis and substitute identification. It is invoked when a user requests a holiday ticket for a named worker.

The first substantive step, after deanonymisation, is an authorisation gate. The system verifies that the caller (Paolo Ricci in the mock data, the hardcoded identity returned when the real identity service is unavailable) has a legitimate relationship with the named worker. Three authorisation paths are accepted: the caller manages the worker's cantiere via a `PULITO_DA` perimeter (i.e., the worker appears in one of the cantieri the caller is authorised for), the caller is the worker's `RESPONSABILE_DI_RIFERIMENTO`, or the caller is the worker's `AUTORIZZATORE`. If none of these conditions holds, the method returns an authorisation failure message without processing the request further.

For each cantiere to which the worker is assigned, the method constructs a per-cantiere absence report. The full worker pool for the cantiere is retrieved, and each worker's absence status for the current date is checked via `AbsenceTools.GetAbsencesForPerson` with the

`UnavailabilityGenerated = true` filter. Workers with active absence tickets overlapping the current date are marked as unavailable. The remaining active workers constitute the available coverage pool for the cantiere.

The substitute identification follows the same Haversine proximity logic as Use Case B, but applied to the geographic context of the cantiere rather than the target person's residence. Non-colleagues within 15km of the cantiere's coordinates who are not currently absent are ranked and presented as potential substitutes. The per-cantiere report includes absence category breakdown (FERIE vs. MALATTIA), the list of absent colleagues, and the ranked substitute pool. A consolidated log file is written to `logs/` and the confirmation string is returned to the client.

4.5 Backend Integration and Data Abstraction

4.5.1 The IMiorelliService Interface

All data access in the JobTools orchestration layer is performed through the `IMiorelliService` interface, which defines the complete data contract required by the business tools:

```

1 public interface IMiorelliService
2 {
3     // Identity
4     Task<MiorelliCallerIdentity> GetCallerIdentityAsync(
5         string? username = null);
6
7     // Assets
8     Task<List<MiorelliCantiere>> GetAuthorizedCantieriAsync(
9         Guid personId, Guid? itemCategoryId = null);
10    Task<List<MiorelliRelation>> GetPeopleByRelationOnAssetAsync(
11        Guid assetId, Guid? relationTypeId = null);
12    Task<List<MiorelliCantiere>> GetAssetsNearCoordinatesAsync(
13        double latitude, double longitude,
14        double maxDistanceMeters);
15
16    // Persons
17    Task<MiorelliPerson?> GetPersonByIdAsync(Guid personId);
18    Task<List<MiorelliPerson>> SearchPersonByNameAsync(
19        string name);
20    Task<List<MiorelliColleague>> GetColleaguesOfPersonAsync(
21        Guid personId, Guid relationTypeId,
22        bool includeDetails = false);
23
24    // Absences
25    Task<List<MiorelliTicket>> GetAbsencesForPersonAsync(
26        Guid personId, long? category = null,
27        DateTimeOffset? from = null,
28        DateTimeOffset? to = null);
29
30    // Lookups
31    Task<List<MiorelliRelationType>> GetRelationTypesAsync(
32        string? shortNameFilter = null);
33    Task<List<MiorelliItemCategory>> GetItemCategoriesAsync(
34        string? shortNameFilter = null);

```

35 }

Listing 4.4: IMiorelliService interface (abridged)

The interface was designed to mirror the expected schema of the real Miorelli REST API, informed by the API documentation available at the time of development. All method signatures use the internal DTO types (`MiorelliPerson`, `MiorelliCantiere`, `MiorelliRelation`, etc.) rather than API response types, ensuring that the mapping between the external API format and the internal representation is encapsulated in the concrete implementation and does not bleed into the tool layer.

This design achieves a strict separation of concerns: `JobTools` expresses its data requirements through the interface contract, with no knowledge of where the data comes from or how it is formatted by the source. The concrete implementation (mock, stub, or the real generated client) is invisible to the orchestration logic.

4.5.2 The Stub/Mock Pattern

At the time of the internship, the real Miorelli tenant had not been provisioned for development access. Rather than blocking development on backend availability, the project adopted a stub/mock pattern that decouples the AI orchestration layer from the backend integration layer.

`MiorelliServiceStub` implements `IMiorelliService` by throwing a `MiorelliClientNotAvailableException` on every method call:

```

1 public class MiorelliClientNotAvailableException : Exception
2 {
3     public MiorelliClientNotAvailableException()
4         : base("Miorelli client unavailable. Using mock data.")
5     { }
6 }
7
8 public class MiorelliServiceStub : IMiorelliService
9 {
10     public Task<MiorelliCallerIdentity> GetCallerIdentityAsync(
11         string? username = null)
12         => throw new MiorelliClientNotAvailableException();
13     // ... same for all other methods
14 }

```

Listing 4.5: MiorelliServiceStub structure

Each tool class that calls `IMiorelliService` wraps its calls in a `try/catch` that specifically catches `MiorelliClientNotAvailableException` and falls back to the corresponding method in `MiorelliMockDataProvider`, a static class containing realistic synthetic data for the Lombardy region. The use of a specific exception type (rather than a general `Exception`) ensures that unexpected errors from the real client propagate fatally rather than being silently absorbed by the fallback path.

The mock dataset is designed to exercise the full complexity of the three use cases: ten persons with GPS coordinates derived from real Lombardy municipalities, three cantieri (Bergamo, Brescia, Mantova) with realistic addresses and coordinates, a set of `PULITO_DA` relations that creates a non-trivial colleague graph, and five absence tickets with dates that produce partially absent cantiere coverage when Use Case C is evaluated. All entity GUIDs in

27

the mock dataset are stable, deterministic values (e.g., 1111000N-0000-0000-0000-000000000000 for person N) so that test outputs are reproducible across runs.

4.5.3 NSwag Client Generation and the Migration Path

The Anonymization API exposes its interface as an OpenAPI specification, served at `GET /swagger/api-docs.yaml` in development mode. This specification is the source of truth for the C# clients generated in both the MCP server and the MCP client. NSwag [13] processes the YAML specification and produces a strongly typed C# client class, `AnonymizationServiceClient.g.cs`, that wraps all three endpoints (`AnonymizeAsync`, `AnonymizeIntoContextAsync`, `DeanonymizeKnownAsync`) in awaitable methods with typed request and response objects. The generated file carries a *do not edit* header; any change to the API surface is reflected by re-running `nswag run nswag.json` from the consuming project.

Migrating from the mock Miorelli data to the real Miorelli production API requires three coordinated changes. First, the `AntaresMiorelliClient` is generated from the Miorelli OpenAPI specification using the same NSwag workflow used for the Anonymization API client. Second, a mapping layer is implemented that translates Miorelli API response types to the internal DTO types defined in the `IMiorelliService` contract. Third, the dependency injection registration in `program.cs` is updated from:

```
1 // Current (stub with mock fallback):
2 builder.Services
3     .AddScoped<IMiorelliService, MiorelliServiceStub>();
4
5 // After tenant provisioning:
6 builder.Services
7     .AddScoped<IMiorelliService, AntaresMiorelliClient>();
```

Listing 4.6: Switching from stub to real Miorelli client

No changes are required in any tool class (`JobTools`, `PersonTools`, `AssetTools`, `AbsenceTools`, `LookupTools`). The interface contract is the stable boundary; the implementation behind it is substitutable without affecting the orchestration logic. The tenant-specific GUIDs currently defined as compile-time constants in `MiorelliMockConstants` should be replaced with real values from the Miorelli admin panel, or loaded from a configuration file to avoid hardcoding values that differ between environments.

“latex

Chapter 5

Use Cases and Validation

- 5.1 Workforce Management Domain Overview
- 5.2 Use Case A: Spatial Querying for Construction Sites
- 5.3 Use Case B: Colleague Identification and Proximity Ranking
- 5.4 Use Case C: Holiday Request and Absence Analysis

Chapter 6

Discussion and Future Work

Chapter 7

Conclusions

7.1 Summary of the Work

7.2 Final Remarks on Enterprise AI Adoption

Bibliography

- [1] Anthropic. *Model Context Protocol Specification*. <https://modelcontextprotocol.io/specification>. Open protocol for connecting LLM clients to external tools and data sources. Accessed: June 2026. 2024.
- [2] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.
- [3] Hendrik Bänder. “Decoupling language and editor — the impact of the language server protocol on textual modelling tools”. In: *Proceedings of the 12th System Analysis and Modelling Conference*. 2019, pp. 74–83.
- [4] Ann Cavoukian. *Privacy by Design: The 7 Foundational Principles*. Tech. rep. Information and Privacy Commissioner of Ontario, Canada, 2009.
- [5] Vincenzo Dalia. “Workflow assistito da AI per la creazione documentale: un approccio conversazionale basato su RAG”. Relatori: Luigi De Russis, Daniele Sabetta; sessione di laurea Aprile 2025. Accessed: July 2026. Tesi di Laurea Magistrale in Ingegneria Informatica. Politecnico di Torino, 2025.
- [6] Jacob Devlin et al. “BERT: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Vol. 1. 2019, pp. 4171–4186.
- [7] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. Official Journal of the European Union, L 119, pp. 1–88. 2016.
- [8] Kai Greshake et al. “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection”. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 2023, pp. 79–90.
- [9] Stefan Larson et al. “An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 1311–1316.
- [10] Daniele Mansillo. “Multimodal RAG for Slide Presentations with Synthetic Data Generation and Anonymization”. Relatori: Daniele Apiletti, Simone Monaco. Accessed: July 2026. Tesi di Laurea Magistrale in Data Science and Engineering. Politecnico di Torino, 2025.

BIBLIOGRAPHY

- [11] Microsoft. *.NET 10*. <https://dotnet.microsoft.com>. Cross-platform application framework used for the MCP server, anonymisation API, and MCP client. Accessed: June 2026. 2025.
- [12] Microsoft. *Presidio — Data Protection and De-identification SDK*. <https://github.com/microsoft/presidio>. Accessed: June 2026. 2020.
- [13] NSwag contributors. *NSwag: The Swagger/OpenAPI toolchain for .NET*. <https://github.com/RicoSuter/NSwag>. Used for automatic C# client generation from OpenAPI specifications. Accessed: June 2026. 2020.
- [14] OpenAI. *Model Card for OpenAI Privacy Filter*. <https://cdn.openai.com/pdf/c66281ed-b638-456a-8ce1-97e9f5264a90/OpenAI-Privacy-Filter-Model-Card.pdf>. Released April 22, 2026 under the Apache 2.0 licence. Sparse Mixture-of-Experts token-classification model for PII detection. Benchmark F1 figures (Table 1, Section 7.4.1, p. 14): 96.0% token-level F1 on PII-Masking-300k (baseline), 97.4% on the OpenAI-corrected version. Accessed: July 2026. 2026.
- [15] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com>. Modern, high-performance web framework for building APIs with Python. Accessed: June 2026. 2018.
- [16] Redis Ltd. *Redis*. <https://redis.io>. In-memory data structure store used for ephemeral session context storage. Accessed: June 2026. 2009.
- [17] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [18] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [19] Jason Wei et al. “Emergent abilities of large language models”. In: *Transactions on Machine Learning Research* (2022).
- [20] Shunyu Yao et al. “ReAct: Synergizing reasoning and acting in language models”. In: *The Eleventh International Conference on Learning Representations*. 2023.
- [21] Urchade Zaratiana et al. “GLiNER: Generalist model for named entity recognition using bidirectional transformer”. In: *arXiv preprint arXiv:2311.08526* (2023).

List of Figures

3.1 Runtime architecture of Antares AI. The client anonymises the user prompt before crossing the MCP boundary. The MCP server routes the anonymised request via Claude and executes the selected business tool server-side, where PII is restored from the Redis context. The client receives only a hardcoded confirmation string. 13

LIST OF FIGURES

List of Tables

3.1	Antares AI runtime services	12
-----	---------------------------------------	----

LIST OF TABLES
